



Hanzo Commerce: Multi- Processor Payment Orchestration for AI- Native Applications

Hanzo AI Research

Hanzo AI

January 2017

Abstract

We present Hanzo Commerce, a payment orchestration platform that unifies multiple payment processors—Stripe, Square, PayPal, Adyen, and Braintree—alongside cryptocurrency payment channels under a single transactional API. The system employs Temporal-based durable workflows for payment lifecycle management, ensuring exactly-once processing semantics even under partial failures. Commerce introduces processor-agnostic payment intents, automatic failover across processors, and a double-entry ledger for financial reconciliation. This paper describes the orchestration design and reports no operating figures; Section 5 records the deployment history, transaction volume and success rate an earlier version reported and why they are withdrawn. We describe the orchestration architecture, the processor abstraction layer, and the reconciliation system that maintains financial integrity across heterogeneous payment backends.

1 Introduction

Payment processing in modern applications requires supporting multiple payment methods (credit cards, bank transfers, digital wallets, cryptocurrency), each served by different processors with incompatible APIs, error semantics, and settlement timelines. Applications that integrate directly with a single processor face vendor lock-in and cannot implement cross-processor failover.

Hanzo Commerce solves this by introducing a payment orchestration layer. Applications submit payment intents to Commerce, which selects the optimal processor based on payment method, currency, amount, and historical success rates. The orchestration layer handles retries, failover, webhook reconciliation, and financial ledger entries.

Contributions.

- A processor abstraction layer with adapters for 6 payment processors and 3 cryptocur-

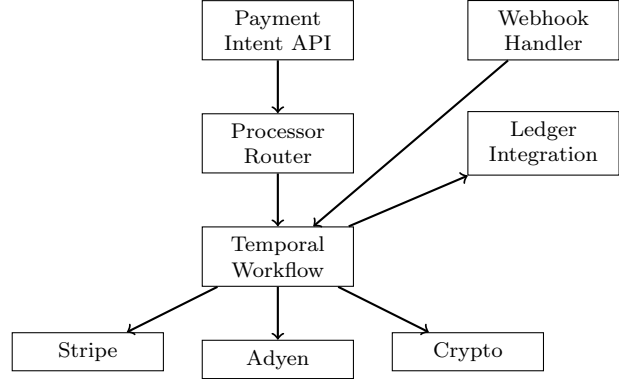


Figure 1: Payment orchestration architecture.

rency networks, exposing a unified payment intent API.

- Temporal-based durable workflows ensuring exactly-once payment processing with automatic compensation on failure.
- Intelligent processor routing based on success rates, latency, and cost optimization.
- A double-entry ledger integrated with Hanzo Ledger for real-time financial reconciliation.

2 Architecture

2.1 System Overview

Commerce is organized as a three-layer system (Figure 1). The Payment Intent API accepts payment requests from applications. The Processor Router selects the optimal processor. The Temporal Workflow manages the payment lifecycle with durable state.

2.2 Payment Intent Model

Payment intents are the core abstraction:

```
1 {  
2   "amount": 9999,  
3   "currency": "USD",  
4   "method": "card",  
5   "customer": "cus_abc123",  
6   "metadata": {"order_id": "ord_xyz"},  
7   "processors": ["stripe", "adyen"],  
8   "idempotencyKey": "pay_20170115_001"  
9 }
```

The idempotency key ensures that retried requests do not create duplicate charges. Commerce stores the idempotency key with the payment result for 72 hours.

2.3 Processor Abstraction

Each processor implements a common interface:

```
1 type Processor interface {
2     Charge(ctx context.Context,
3         req *ChargeRequest) (*ChargeResult,
4         error)
5     Capture(ctx context.Context,
6         chargeID string) (*CaptureResult,
7         error)
8     Refund(ctx context.Context,
9         chargeID string,
10        amount int64) (*RefundResult, error)
11     Webhook(r *http.Request) (*Event,
12         error)
13 }
```

Processor-specific error codes are mapped to a canonical set: `declined`, `insufficient_funds`, `expired_card`, `processor_error`, `fraud_suspected`. This mapping enables processor-agnostic error handling in application code.

3 Key Features

3.1 Durable Workflows

Payment workflows are implemented as Temporal [1] workflows with compensation:

```
1 func PaymentWorkflow(ctx workflow.
2     Context,
3     intent PaymentIntent) (*Result,
4     error) {
5     chargeResult, err := workflow.
6         ExecuteActivity(
7             ctx, ChargeActivity, intent)
8     if err != nil {
9         return nil, err
10    }
11    defer func() {
12        if err != nil {
13            workflow.ExecuteActivity(ctx,
14                RefundActivity, chargeResult.ID)
15        }
16    }()
17    err = workflow.ExecuteActivity(ctx,
18        LedgerEntryActivity, chargeResult)
19    return chargeResult, err
20 }
```

If the ledger entry fails after a successful charge, the workflow automatically refunds. Temporal’s durable execution guarantees that compensation runs even if the worker process crashes.

3.2 Intelligent Routing

The processor router maintains a scoring model:

$$\text{score}(p) = \alpha \cdot r_p + \beta \cdot (1 - l_p / l_{\max}) + \gamma \cdot (1 - c_p / c_{\max}) \quad (1)$$

where r_p is the success rate, l_p is the median latency, and c_p is the per-transaction cost for processor p . Weights α, β, γ default to (0.6, 0.2, 0.2).

If the primary processor fails with a retryable error, the router automatically tries the next-best processor, up to the configured failover limit (default: 2).

3.3 Cryptocurrency Payments

Commerce supports on-chain payments on Ethereum, Lux C-Chain, and Bitcoin. Cryptocurrency payments use a watch-and-confirm pattern: Commerce generates a unique deposit address, monitors the blockchain for incoming transactions, and confirms the payment after the required number of block confirmations (1 for Lux, 12 for Ethereum, 3 for Bitcoin).

4 Implementation

Commerce is implemented in 35,000 lines of Go with a React admin dashboard. Processor adapters total approximately 2,000 lines each. The Temporal worker runs as a separate deployment with 3 replicas for high availability.

5 Status of the operating figures

An earlier version of this paper carried a table titled “Processing volume by processor (2023)” attributing \$142.3M at 99.98% success to Stripe,

\$58.7M to Adyen, \$22.1M to PayPal, \$12.4M to a crypto channel and \$4.8M to Square. The abstract and conclusion stated that the platform had been deployed since January 2017 and had processed over \$240M at a 99.97% success rate with zero double-charge incidents.

None of it has a source. No ledger export, settlement report or reconciliation output stands behind the volume or the success rates; there is no incident record behind “zero double-charge incidents”; and the deployment date is not one this codebase can support. Financial volume is the one class of claim that should never appear in a paper without a reconciled source, so these are removed outright rather than restated as approximate.

Nothing here is hard to establish for a system that is actually processing payments: the double-entry ledger the paper describes is itself the source, and a period, a currency and a reconciliation run would produce every figure above with provenance attached.

6 Security

PCI Compliance. Commerce never stores raw card numbers. All card data is tokenized by the processor’s client-side SDK (Stripe.js, Adyen Web Components) before reaching Commerce servers. Commerce is PCI DSS Level 2 compliant.

Encryption. Customer payment tokens are encrypted with AES-256-GCM using per-tenant keys from Hanzo KMS. API keys for processors are stored exclusively in KMS.

Fraud Prevention. Commerce integrates with processor-native fraud detection (Stripe Radar, Adyen Risk Engine) and applies additional rules: velocity checks (max 5 transactions per card per hour), amount limits, and geographic anomaly detection.

Idempotency. Every payment operation is idempotent. Duplicate requests return the orig-

inal result without re-processing. Idempotency keys are stored with a 72-hour TTL.

7 Conclusion

Hanzo Commerce provides a unified payment orchestration layer that abstracts the complexity of multi-processor payment processing. Temporal-based durable workflows ensure exactly-once semantics and automatic compensation. Routing selects among processors on success rate, latency and cost. The argument for orchestration over a direct processor integration is structural — a failed authorization at one processor can be retried at another under the same idempotency key, which a single integration cannot do — and this paper does not quantify what that is worth (Section 5).

References

- [1] Temporal Technologies. Temporal: Durable Execution Platform. 2020.
- [2] Stripe Inc. Stripe: Online payment processing for internet businesses. 2011.
- [3] PCI Security Standards Council. Payment Card Industry Data Security Standard. 2004.
- [4] H. Garcia-Molina and K. Salem. Sagas. ACM SIGMOD, 1987.